UCS761 - Deep Learning Lab 6

From Shallow to Deep: What Really Changes?

What You Must Already Know

Before starting, you should be able to answer:

- What is a neuron?
- What is a layer?
- What are weights?
- What is bias?
- What is activation?
- What is gradient?
- What changes during learning?

If you cannot answer these, revise.

If you cannot explain:

$$z = w^\top x + b$$

and

$$w := w - \eta \frac{\partial L}{\partial w}$$

you are not ready for this lab.

Dataset (Same for All)

We use a controlled nonlinear dataset.

```
importnumpyasnp

np.random.seed(42)

X=np.random.uniform(-2,2, (400,3))

y= (
np.sin(X[:,0])+
0.5* (X[:,1]**2)-
0.8*X[:,2]
)

y=y.reshape(-1,1)
```

Why?

- Nonlinear
- Smooth

- Requires bending

The sine term forces oscillation.

The quadratic term forces curvature.

The linear term ensures mixed behavior.

A linear model cannot perfectly fit this function.

Architectures To Build

You must build FOUR architectures.

Model A - Shallow

$$3 \to 4 \to 1$$

Hidden activation: ReLU

Output: Linear

Model B - Medium

$$3 \to 6 \to 6 \to 1$$

Hidden activation: ReLU

Model C - Deep

$$3 \to 8 \to 8 \to 8 \to 8 \to 1$$

(4 hidden layers)

Hidden activation: ReLU

Model D - Very Deep

$$3 \to 8 \to 8 \to 8 \to 8 \to 8 \to 8 \to 8 \to 8 \to 1$$

(8 hidden layers)

You must run this twice:

- With Sigmoid
- With ReLU

Now we are at real depth.

Depth = number of sequential transformations.

Each additional layer multiplies gradients again.

“Input dimensions = 3”

Parameter Counting (Compulsory Before Coding)

You must calculate total parameters manually.

Example for:

$$3 \to 8 \to 8 \to 1$$

Layer 1:

$$8 \times 3 + 8 = 32$$

Layer 2:

$$8 \times 8 + 8 = 72$$

Output:

$$1 \times 8 + 1 = 9$$

Total:

$$32 + 72 + 9 = 113$$

You must calculate for all models.

Parameter count formula for any layer:

$$\text{Parameters} = (\text{neurons}_{out} \times \text{neurons}_{in}) + \text{neurons}_{out}$$

No coding until this is written.

Activation Functions To Implement

You must implement:

- ReLU
- Sigmoid
- Tanh
- Leaky ReLU
- Softplus

With derivatives.

No shortcuts.

Definitions:

ReLU:

$$\text{ReLU}(z) = \max(0, z)$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Leaky ReLU:

$$\text{LeakyReLU}(z) = \begin{cases} z & z > 0 \\ \alpha z & z \leq 0 \end{cases}$$

Softplus:

$$\text{Softplus}(z) = \log(1 + e^z)$$

You must also implement their derivatives.

Initialization Rules

Weights:

$$[-0.5, 0.5]$$

Biases: Zeros.

Learning rate:

$$\eta = 0.01$$

Epochs: 1000

Loss:

$$L = \frac{1}{n} \sum (y - \hat{y})^2$$

We use MSE here instead of BCE because this is regression.

The Critical Measurement

For every model:

You must compute gradient norm for:

- First hidden layer
- Last hidden layer

Compute:

$$\|G\| = \sqrt{\sum G^2}$$

Print both at final epoch.

This is L2 norm of gradients.

You are measuring gradient magnitude.

This lab is about observing gradient behavior.

What You Must Record

For each model:

- Final Loss
- Loss at epoch 200
- Gradient norm (Layer 1)
- Gradient norm (Last hidden layer)
- Training behavior (stable / slow / unstable)

No explanations yet.

Only record.

Deep Model Experiment

For Model D (8 hidden layers):

Run:

- Once with ReLU
- Once with Sigmoid

Compare:

- Final loss
- Gradient norm at first layer
- Gradient norm at last layer

Do not explain.

Just record.

Observation Table

You must submit:

Model	Activation	Final Loss	Grad Norm L1	Grad Norm Last	Observation

Reflections

Answer:

- Did deeper always reduce loss faster?
- Did gradients in early layers stay similar to later layers?
- Was training equally stable for all activations?
- Which activation behaved more stable in deeper networks?
- Did some models improve very slowly even though learning rate was same?

Do not search online.

Only based on observation.

Minimal Skeleton Required (Must Generalize)

Forward structure (generic layer):

$$Z = W@A_prev + bA = activation(Z)$$

Backward structure:

$$dZ = dA * activation_derivative(Z) dW = (1/N) * (dZ@A_prev.T) db = (1/N) * np.sum(dZ, axis = 1, keepdims$$

Students must generalize this across L layers.

No hardcoding per layer allowed.